

Linguagens Formais e Autômatos

Prof. Yandre Costa

12 de abril de 2026

Site do Professor: <http://www.din.uem.br/yandre>

1 Teoria da Computação

→ Ciência da Computação

- Ênfase teórica: ideias fundamentais e modelos computacionais.
- Ênfase prática: projeto de sistemas computacionais.

→ A teoria da computação possui duas grandes vertentes.

1. Linguagens Formais e Autômatos.
2. Máquinas Universais e Computabilidade.

→ O que é Teoria da Computação?

- Estudos teóricos acerca da capacidade de resolução de problemas das máquinas.
- Estudo de modelos formais que:
 - * Caracterizam em nível conceitual: programas, máquinas e enfim, a computação.
 - * Especificam o que é computável ou não.
 - * Ajudam na especificação de linguagens artificiais entre outras aplicações.

→ Histórico da Computação

- Computar, do latim "computare", que significa calcular, avaliar, contar.

2 Conceitos Básicos

• Introdução

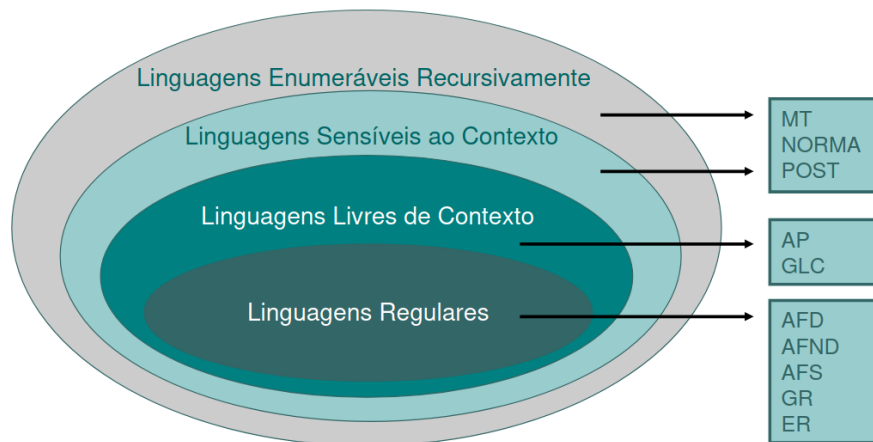
- Problema: definir um conjunto de cadeias de símbolos
- Exemplo: conjunto M dos números binários que têm 2 dígitos

$$M = \{00, 01, 10, 11\}$$

- Porém, se fosse o conjunto N de todas as combinações de dígitos binários, poderíamos tentar o seguinte:

$$N = \{0, 1, 00, 01, 10, 11, 000, \dots\}$$

- * Percebe-se que este conjunto é infinito
- Representação clara: humanos x computador
- Representação formal: computador
- Um objetivo de LFA é estudar uma maneira precisa e formal de descrever sequências de símbolos pertencentes a um determinado conjunto.
- Em especial conjuntos que não podem ser trivialmente enumerados.
- Os estudos iniciais foram em torno de Linguagens Naturais (LN).
- Algumas características de LN introduziram dificuldades no tratamento computacional das mesmas.
 - * LN é extensa, complexa, não tem sintaxe rígida e semântica bem determinada (rica em ambiguidade)
- As maneiras sistêmicas de descrever uma linguagem de programação são:
 - * Um método que permite construir programas sintaticamente corretos - geração (Gramática)
 - * Um método que permite verificar se um programa escrito está sintaticamente correto - reconhecimento (Autômatos)
- Hierarquia de Chomsky



- Pesquisas já demonstraram o uso destas técnicas em:
 - * Análise de linguagens de programação
 - Léxica
 - Sintática
 - * Modelos de sistemas biológicos
 - * Desenho de hardware
 - * Relacionamentos com linguagens Naturais
 - * Reconhecimento de padrões (em visão computacional por exemplo)

- Alfabeto
 - Conjunto de simbolos finito e não vazio
 - Normalmente denotado por Σ
 - Exemplos
 - * $\Sigma = \{a, b\}$
 - * $\Sigma = \{1, 2, 3\}$
 - * $\Sigma = \{00, 11\}$
- Simbolo ou letra
 - É todo elemento pertencente a um alfabeto
 - a é um simbolo de Σ
- Cadeia ou palavra
 - É uma concatenação de simbolos do alfabeto justapostos.
 - Assim, dado um alfabeto Σ e uma sequencia de simbolos $x = a_1a_2a_3 \dots a_n$, x é uma cadeia sobre Σ se, e somente se, $a_i \in \Sigma$ para $i = 1, 2, \dots, n$.
 - Comprimento de Cadeia ou Tamanho de Palavra
 - * É o numero de simbolos que compoem uma dada cadeia (ou palavra)
 - * O comprimento de uma cadeia x é denotado por $|x|$
 - * Então, a cadeia $x = a_1a_2a_3 \dots a_n$ tem seu comprimento $|x| = n$
 - * Cadeia nula ou palavra vazia: é o caso especial, ela é denotada por λ (ou ϵ) e tem o tamanho igual a zero
 - * Exercício: dado o alfabeto $\Sigma = a, b, c, de$, verifique se as cadeias a seguir são formadas sobre este alfabeto, e se for, verifique qual o comprimento
- Exponenciação de Alfabetos
 - Σ^k é o conjunto de todas as cadeias com tamanho k formadas sobre o alfabeto Σ
 - Exemplo: Considere $\Sigma = \{0, 1\}$
 - * $\Sigma^0 = \lambda$
 - * $\Sigma^1 = \{0, 1\}$
 - * $\Sigma^2 = \{00, 01, 10, 11\}$
 - * $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$
- Fechamento de um Alfabeto
 - Seja Σ um alfabeto, então, o fechamento de Σ descrito por Σ^* é definido como

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots \cup \Sigma^n \cup \dots$$
- Concatenação de cadeias
 - Dado o alfabeto Σ e as cadeias $x, y \in \Sigma^*$, a concatenação de x e y , indicada por xy , produz uma cadeia formada pelos simbolos de x seguidos pelos simbolos de y .

- Se $x = a_1a_2a_3 \dots a_n \in \Sigma^*$ e $y = b_1b_2b_3 \dots b_m \in \Sigma^*$ então:

$$xy = a_1a_2a_3 \dots a_nb_1b_2b_3 \dots b_m$$

- Exemplos:

$$\Sigma = \{a, b\}$$

$$x = abaa, y = ba, z = \lambda$$

$$xy = abaaba$$

$$yx = baabaa$$

$$yz = ba = zy = y$$

- A cadeia nula (λ) é o elemento neutro da concatenação
- Concatenação Sucessiva – Concatenação de uma palavra com ela mesma.
 - * Representada através de um expoente: w^n – Onde w é uma palavra e n indica o número de concatenações sucessivas.

- Dado um alfabeto Σ e $x, y \in \Sigma^*$, diz-se que:

- x é um prefixo de y , se e somente se, $\exists w \in \Sigma^*$ tal que $y = xw$
- x é um sufixo de y , se e somente se, $\exists w \in \Sigma^*$ tal que $y = wx$
- x é uma subpalavra de y , se e somente se, $\exists w, u \in \Sigma^*$ tal que $y = wxu$

- Linguagem

- Dado o alfabeto Σ , o conjunto de palavras L é uma linguagem sobre Σ se e somente se $L \subset \Sigma^*$.

- Operações sobre linguagens:

- * Considere L_1 e L_2 linguagens definidas sobre Σ

- União: $L_1 \cup L_2 = \{x | x \in L_1 \vee x \in L_2\}$
- Interseção: $L_1 \cap L_2 = \{x | x \in L_1 \wedge x \in L_2\}$
- Diferença: $L_1 - L_2 = \{x | x \in L_1 \wedge x \notin L_2\}$
- Concatenação: $L_1L_2 = \{x | x = yz, y \in L_1 \wedge z \in L_2\}$
- Complemento: $\bar{L}_1 = \{x | x \in \Sigma^* \wedge x \notin L_1\}$

- Comparando as definições:

- Linguagem Natural:

- * Uma palavra em português equivale a um símbolo
- * Uma sentença da língua portuguesa é uma cadeia composta de vários símbolos.

- Linguagem computacional

- * Cada programa escrito numa linguagem computacional corresponde a uma cadeia de símbolos que podem ser:
 - Identificadores;
 - Palavras reservadas;
 - Símbolos especiais e operadores;
 - Constantes numéricas.

3 Autômatos Finitos

- AFD – Modelo matemático para definição de linguagem
- Caráter reconhecedor
- Modelam também sistemas de estados finitos
- Exemplo clássico: problema HLCR (Hopcroft e Ullman)

- Problema: um homem quer atravessar um rio levando consigo um lobo, uma cabra e um repolho, e no bote, só cabem ele e mais um dos outros três.
- Exemplos de possíveis estados do sistema:
 - $\langle \text{HLCR-0} \rangle$ – todos na margem esquerda
 - $\langle \text{L-HCR} \rangle$ – lobo na margem esquerda, cabra e repolho na direita
- Entradas do sistema:
 - h – Homem atravessa o rio sozinho
 - l – Homem atravessa o rio com o lobo
 - c – Homem atravessa o rio com a cabra
 - r – Homem atravessa o rio com o repolho
- Objetivo: $\langle \text{HLCR-0} \rangle \Rightarrow \langle \text{0-HLCR} \rangle$
- Representação por diagrama:
 - Círculos representam estados
 - Arcos representam ação ou transição (de um estado para o outro)
 - O estado final é marcado por um círculo duplo
 - As respostas para o problema são as sequências de ações que levam do estado inicial para o final

3.1 Autômato Finito Determinístico

- Exemplo de sistema que pode ser representado desta forma:
 - Forno de micro-ondas
 - * Entradas: porta aberta ou fechada, comandos fornecidos pelo cozinheiro através do painel, sinal do "timer" que expira
 - * Estados: aberto, esperando por comandos, cozinhando, desligado
- Definições Básicas
 - Um AFD A define uma linguagem $L(A)$ sobre um alfabeto Σ
 - Caráter reconhecedor, ao contrário das gramáticas estudadas que tinham caráter gerador

- * Dada uma cadeia x , ela pertence a $L(A)$
- Uma abstração de um AFD
 - * Uma cabeça de leitura extrai sequencialmente o conteúdo de uma fita (string)
 - * Uma luz de aceitação que acende somente se a cadeia pertencer a linguagem representada pela AFD
 - * Exemplos de cadeias aceitas em HLCR
 - chrcrlhc, ccchllrclllhccc, ...
- Definição Matemática de um AFD
 - * Uma AFD é uma quintupla $\langle \Sigma, S, S_0, \delta, F \rangle$, onde:
 - Σ é o alfabeto de Entradas
 - S é um conjunto finito não vazio de estados
 - S_0 é o estado inicial, $S_0 \in S$
 - δ é a função de transição de estados, definida $\delta : S \times \Sigma \rightarrow S$
 - F é o conjunto de estados finais, $F \subseteq S$
 - * Uma cadeia x para ser aceito, deve levar do estado S_0 para algum estado pertencente a F
 - * A função δ determina como são as transições de estados. Ela leva um par $\langle s, a \rangle$ onde s é um estado e a uma letra do alfabeto num estado s'
 - $\delta(s, a) = s'$
 - * Então, dado um AFD $A = \langle \Sigma, S, S_0, \delta, F \rangle$ e a cadeia $x = a_1 a_2 a_3 \dots a_n \in \Sigma^*$, o autômato parte de S_0 . Ao processar a_1 passa para o estado $\delta(S_0, a_1)$, ao processar a_2 passa para o estado $\delta(S_0, a_2)$, e assim por diante até processar a_n . Nesse ponto, o autômato estará num estado R qualquer. Se $R \in F$, então a cadeia $x \in L(A)$.
 - * Finito: número de estados envolvidos no sistema é finito
 - * Determinístico: Estabelece que para uma cadeia $x \in L(A)$, só existe uma única sequência de estados no AFD A para processá-la.

• Exercícios

- Dados os seguintes autômatos:
- Identifique a linguagem associada a cada um
 - R: Autômato 1: $L = \{a^n b c^m \vee a^n c b^m | n \geq 0, m \geq 0\}$
 - Autômato 2: $L = \{a a^n b b^m | n \geq 0, m \geq 0\}$
 - Autômato 3: $L = \{a..z A..Z(a..z A..Z 0..9)^n | n \geq 0\}$
- Descreva as funções de transição de cada um (exceto para o terceiro diagrama)
- R: Autômato 1:

$$\delta(S_0, a) = S_0; \delta(S_0, b) = S_1; \delta(S_1, c) = S_1$$

OU

$$\delta(S_0, a) = S_0; \delta(S_0, c) = S_2; \delta(S_2, b) = S_2$$

Autômato 2:

$$\delta(S_0, a) = S_1; \delta(S_1, a) = S_1; \delta(S_1, b) = S_2; \delta(S_2, b) = S_2$$

– Exemplo:

- * Modelagem de uma "vending machine" que aceita moedas de 5, 10 e 25 centavos. O preço do produto que ela entrega é 30 centavos.
- * Partindo do estado inicial (0 centavos), deveremos reconhecer sequencias que nos levem a estados finais (valor inserido ≥ 30 centavos)
- * Podemos chamar o autômato de V

* Assim:

- $V = \langle \Sigma, S, S_0, \delta, F \rangle$ onde:
- $\Sigma = \{5, 10, 25\}$ – Cada um destes símbolos (ou letras) representa uma ação.
- $S = \{ \langle 0c \rangle, \langle 5c \rangle, \langle 10c \rangle, \langle 15c \rangle, \langle 20c \rangle, \langle 25c \rangle, \langle 30c \rangle, \langle 35c \rangle, \langle 40c \rangle, \langle 45c \rangle, \langle 50c \rangle \}$ – Cada estado indica quanto foi depositado.
- $S_0 = \langle 0c \rangle$ – estado inicial.
- $F = \{ \langle 30c \rangle, \langle 35c \rangle, \langle 40c \rangle, \langle 45c \rangle, \langle 50c \rangle \}$ – estado onde a entrada é válida e o produto pode ser liberado.
- Delta é definido como:

$$\begin{aligned}
 \delta(\langle 0c \rangle, 5) &= \langle 5c \rangle \\
 \delta(\langle 0c \rangle, 10) &= \langle 10c \rangle \\
 \delta(\langle 0c \rangle, 25) &= \langle 25c \rangle \\
 \delta(\langle 5c \rangle, 5) &= \langle 10c \rangle \\
 \delta(\langle 5c \rangle, 10) &= \langle 15c \rangle \\
 \delta(\langle 5c \rangle, 25) &= \langle 30c \rangle \\
 \delta(\langle 10c \rangle, 5) &= \langle 15c \rangle \\
 &\dots
 \end{aligned}$$

- Tabela de transição de estados

δ	5	10	25
$\langle 0c \rangle$	$\langle 5c \rangle$	$\langle 10c \rangle$	$\langle 25c \rangle$
$\langle 5c \rangle$	$\langle 10c \rangle$	$\langle 15c \rangle$	$\langle 30c \rangle$
$\langle 10c \rangle$	$\langle 15c \rangle$	$\langle 20c \rangle$	$\langle 35c \rangle$
$\langle 15c \rangle$	$\langle 20c \rangle$	$\langle 25c \rangle$	$\langle 40c \rangle$
$\langle 20c \rangle$	$\langle 25c \rangle$	$\langle 30c \rangle$	$\langle 45c \rangle$
$\langle 25c \rangle$	$\langle 30c \rangle$	$\langle 35c \rangle$	$\langle 50c \rangle$
$\langle 30c \rangle$	-	-	-
$\langle 35c \rangle$	-	-	-
$\langle 40c \rangle$	-	-	-
$\langle 45c \rangle$	-	-	-
$\langle 50c \rangle$	-	-	-

A tabela de transição é boa para demonstrar transições grandes, e também para implementar em codigos, usando uma matriz para descrever a tabela de transição!!

Para descrevermos que a quantia de itens é a definição de um AFD, podemos dizer, por exemplo, que $L = \{x \in \Sigma^* \mid x|_a \text{ condição} \}$

3.2 Autômato Finito Não Determinístico – AFND

- Modelo matemático semelhante ao AFD
- Condições mais flexíveis
- Podem haver múltiplos caminhos para processar uma cadeia
- Definição formal
 - Uma AFD é uma quintupla $\langle \Sigma, S, S_0, \delta, F \rangle$, onde:
 - * Σ é o alfabeto de entradas
 - * S é um conjunto finito não vazio de estados
 - * S_0 é um conjunto não vazio de estados iniciais, $S_0 \subseteq S$
 - * δ é a função de transição de estados, definida $\delta : S \times \Sigma \rightarrow \rho(S)$
 - A função pode ativar todos os possíveis subconjuntos dos estados
 - * F é o conjunto de estados finais, $F \subseteq S$
 - Σ, S e F são os mesmos do AFD;
 - S_0 era um único estado em AFD. Em AFND é um conjunto com pelo menos um estado inicial
 - * Então em AFD, $S_0 \in S$, e em AFND, $S_0 \subseteq S$
 - Um AFND pode ter mais de um estado "ativo" (corrente) num instante;
 - Inicialmente, todo estado de S_0 são ativados;
- "Alternativas" para processar uma única cadeia;
- Se o processamento de uma cadeia não levar ao estado final por um caminho, deve-se tentar por outros caminhos (se houverem);
- Se algum dos caminhos possíveis levar a cadeia x a um estado final, então ela faz parte da linguagem definida pelo automato.
- A cadeia é rejeitada se a partir de nenhum estado inicial for possível atingir um estado final ao término do processamento;
- A função δ agora é definida por $S \times \Sigma \rightarrow \rho(S)$
- Exemplo da função $\delta(S, a) = R, T$
imagem
- Se o estado S estiver ativo e a letra a for processada, então tanto R quanto T passaram a estar ativos;
- Se seguirmos por R e não chegarmos a um estado final, podemos tentar por T
- Se alguma das alternativas levar a um estado final, a cadeia é reconhecida
- Se nenhuma alternativa levar a um estado final, a cadeia é rejeitada;
- Ao processar uma letra, todas as transições rotuladas com aquela letra, a partir de todos os estados ativos serão efetuadas
- Então, podemos novamente observar que podemos ter vários estados ativos ao mesmo tempo, ao contrário dos AFDs

3.3 Equivalência entre AFD e AFND

- A classe dos AFDs é equivalente à classe dos AFNDs. Assim, para todo AFD existe um AFND equivalente e vice-versa.
- Exemplo – $L(A) = \{w | w \in \{a, b\}^* \wedge w \text{ possui } aaa \text{ como sufixo}\}$
imagem
- Exercício
 - Construa um AFD e um AFND para a seguinte linguagem:
 - * $L(A) = \{w | w \in a, b^* \wedge w\}$ possui aa como subpalavra
imagem
- Transformação de AFND em AFD
 - Para todo AFND existe um AFD equivalente
 - Dado um AFND $A = \langle \Sigma, S, S_0, \delta, F \rangle$, define-se o AFD $A^d = \langle \Sigma, S^d, S_0^d, \delta^d, F^d \rangle$ equivalente da seguinte forma:
 - * $S^d = \rho(S)$
 - * $S_0^d = S_0$
 - * F^d : Todas as combinações de estados de S^d que possui como componente algum estado de F
 - * Seja $Q \in S^d$, $\delta^d(Q, a)$ vai levar à um estado que corresponde à combinação de todos os estados que podem ser alcançados ao processar o símbolo 'a' a partir de qualquer componente Q
 - Exemplo 1:
 - * Dado o AFND:
imagem
 - * Cria um estado para cada combinação possível de estados do AFND
 - * Estabelece-se como estado inicial a combinação correspondente ao conjunto de estados iniciais do AFND
 - * Estados finais: todos que possuem em sua combinação algum dos estados finais do AFND
 - * Criação das transições:
 - $\delta^d(Q, a)$ será igual ao estado que corresponda à combinação de todos os estados que formam o conjunto $\delta(Q, a)$
imagem
 - Durante a transformação de um AFND em um AFD podem ser criados estados que fiquem "isolados" do(s) estado(s) inicial(is)
 - Exercício: Transforme o AFND descrito a seguir em um AFD
imagem
- Minimização de AFD
 - Dois AFDs A e B são equivalentes, denotado por $A \equiv B$, se $L(A) = L(B)$

- Um automato minimo para uma linguagem regular L é um automato com o menor numero de estados possivel que aceita L
 - * Um AFD $A = \langle \Sigma, S_A, S_{0A}, \delta_A, F_A \rangle$ é dito minimo se para qualquer AFD $B = \langle \Sigma, S_B, S_{0B}, \delta_B, F_B \rangle$ tal que $A \equiv B$, temos que $|S_A| \leq |S_B|$
- Pré-requisitos para a minimização:
 - * O automato deve ser Deterministico
 - * Não pode ter estados inacessiveis a partir do estado inicial
 - * A função de transição deve ser total (todas as saídas devem ser previstas para todos os estados)
 - Para este ultimo requisito muitas vezes é necessario introduzir um estado S_{trash} para lançar as transições não previstas originalmente
- A estratégia consiste em fundir estados equivalentes num mesmo estado
 - * Dois estados são equivalentes se, ao processarem uma mesma cadeia, ambos chegam a estados finais, ou ambos chegam a estados não finais
- Para isto, são identificados estados não-equivalentes e, por exclusão, encontra-se os equivalentes
- Utiliza-se uma tabela triangular que possui um cruzamento para cada par de estados distintos do automato (incluindo S_{trash} quando este é acrescentado)
- Supondo um autômato com $S = \{S_0, S_1, \dots, S_{n-1}, S_n, S_{trash}\}$

imagem

 - * Estados não equivalentes serão sinalizados, ao final do processo, os cruzamentos em branco deverão indicar estados equivalentes.
- Inicia-se a marcação pelos estados trivialmente não-equivalentes
 - * Dois estados não trivialmente não-equivalentes se um é final e o outro não
- Exemplo:
 - * Considere o seguinte AFD e sua tabela de cruzamento de estados

imagem

...
- Critério usado para a marcação dos estados não equivalentes:
 - * Para cada par $\{S_u, S_v\}$ não marcado e para cada simbolo $a \in \Sigma$, suponha que $\delta(S_u, a) = R_u$ e $\delta(S_v, a) = R_v$
 - Se $R_u = R_v$, então S_u e S_v são equivalentes e, para o simbolo a não deve ser marcado
 - Se $R_u \neq R_v$, e o par $\{R_u, R_v\}$ não está marcado, então $\{S_u, S_v\}$ é incluído em uma lista a partir de $\{R_u, R_v\}$ para posterior analise
 - Se $R_u \neq R_v$, e o par $\{R_u, R_v\}$ está marcado, então:
 - $\{S_u, S_v\}$ é não equivalente e deve ser marcado
 - Se $\{S_u, S_v\}$ encabeça uma lista de pares, então todos os pares da lista devem ser marcados
- Na unificação dos estados equivalentes:
 - * Conjuntos de estados finais equivalentes podem ser unificados como um unico estado final

- * Conjuntos de estados não-finais equivalentes podem ser unificados como um unico estado não-final
- * Se algum dos estados equivalentes é inicial, então o correspondente estado unificado é inicial

4 Expressões Regulares

- Formalismo de carater gerador
 - A partir deee pode-se inferir como construir as cadeias da linguagem
 - Pode representar qualquer LR – Linguagem Regular
 - Uma ER é definida a partir de conjuntos básicos e operadores de concatenação e união
 - Formalismo adequado para comunicação homem $\times$ homem e homem $\times$ máquina
 - Uma ER sobre um alfabeto Σ é definida como segue:
 - $\emptyset$ é uma ER e denota a linguagem vazia
 - ε é uma ER e denota a linguagem contendo exclusivamente a palavra vazia, ou seja, $\{\varepsilon\}$
 - Qualquer simbolo x pertence a Σ é uma ER e denota a linguagem contendo a palavra x , ou seja, $\{x\}$
 - Se r e s são ERs e denotam as linguagens R e S , respectivamente, então:
 - * $(r + s)$ é ER e denota a linguagem $R \cup S$ (União)
 - * (rs) é ER e denota a linguagem $RS = \{uv | u \in R \text{ e } v \in S\}$ (Concatenação)
 - * (r^*) é ER e denota a linguagem R^* (Fecho)
 - Pode-se utilizar parenteses ou não, mas deve-se considerar a seguinte convenção:
 - * A concatenação sucessiva (fechamento ou fecho de Kleene) tem precedencia sobre a concatenação
 - * A concatenação tem precedência sobre a união
 - Exemplos de ER
- Tabela
- ERs podem ser usadas em buscas em editores de texto